THEFOCUS.AI LABS

Umwelten

A TypeScript framework for building, hosting, and evaluating AI agents across multiple platforms. This document inventories every feature module, what works, what's partial, and what's missing.

DOCUMENT TYPE: Feature Inventory & Technical Overview

DATE: April 2026

STATUS KEY: ✓ Implemented · ⚠ Partial · ✗ Missing

53K
LINES OF TYPESCRIPT

394
COMMITS

13
MONTHS ACTIVE

10
LLM PROVIDERS

20
FEATURE MODULES

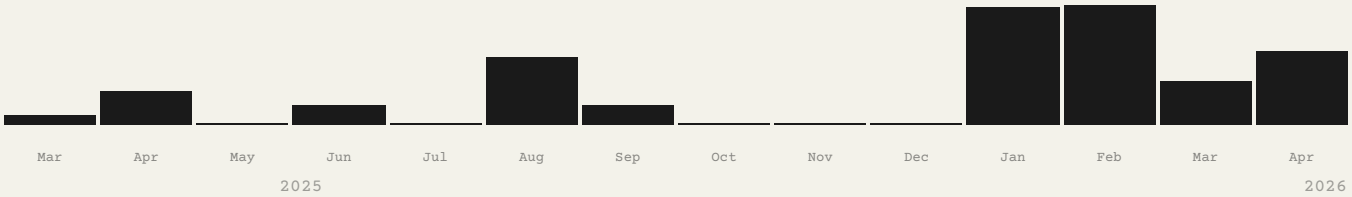
15K
LINES OF TESTS

First commit: **March 2025** · Active development days: **75** · Latest: **April 2026**

00 PROJECT AT A GLANCE

Development Activity

Commits per month — March 2025 to April 2026:



Module Sizes (lines of TypeScript, excluding tests)

Module Timeline

When each module first appeared in the codebase:

MODULE	STARTED	SOURCE LINES	TEST LINES	FILES
providers	Apr 2025	1,533	1,021	13
cli	Apr 2025	7,114	579	13
evaluation	Apr 2025	8,375	3,433	34
memory	Apr 2025	567	104	5
markdown	Apr 2025	463	—	4
cognition	Jun 2025	1,942	1,493	9
stimulus	Jun 2025	3,425	2,102	25
interaction	Jun 2025	6,918	1,880	20
mcp	Jun 2025	2,869	—	6
schema	Aug 2025	1,036	851	10
ui	Aug 2025	5,478	1,322	19
reporting	Jan 2026	1,274	—	7
context	Jan 2026	342	61	8
habitat	Feb 2026	10,403	1,916	46
Total		**52,664**	**15,060**	**221**

PART I — FOUNDATION

00 PROVIDERS — LLM BACKENDS

Unified access to 10 providers via `BaseProvider` (abstract class with `listModels()` and `getLanguageModel()`).

✓ **Google (Gemini)** — native thinking support

✓ **OpenRouter** — access to OpenAI, Anthropic, Meta, etc.

✓ **Ollama** — local models, no API key needed

✓ **LM Studio** — local REST API, no key needed

✓ **GitHub Models** — via GITHUB_TOKEN

✓ **DeepInfra, Together AI, Fireworks, MiniMax, Nvidia**

✓ **Provider registry** — `getModel()`, `validateModel()`, `getModelProvider()`

✓ **Model discovery and search** — `getAllModels()`, `searchModels()`, `findModelByIdAndProvider()`

× **Model fallback/retry across providers** — if a provider is down, no automatic fallback

× **Spend caps/budget limits** — can track costs but can't enforce limits

00 COGNITION — MODEL RUNNERS

How to call LLMs. Wraps Vercel AI SDK with rate limiting, cost tracking, retries.

- ✓ **BaseModelRunner** — `generateText`, `streamText`, `generateObject`, `streamObject`
- ✓ **SmartModelRunner** — before/during/after hooks for intercepting model calls
- ✓ **MemoryRunner** — SmartModelRunner that auto-extracts facts after each response
- ✓ **Rate limiting** — per-model, built into BaseModelRunner
- ✓ **Cost tracking** — automatic per-call calculation with `TokenUsage` and `CostBreakdown`
- ✓ **Usage extraction** — normalize input/output/total tokens across providers
- ✓ **Step assembler** — multi-step tool calling orchestration
- ✓ **Message normalization** — normalize messages across provider formats
- ✓ **Per-user tracking** — `interaction.userId` forwarded to OpenRouter/Anthropic analytics
- ✓ **Reasoning effort control** — `none`, `low`, `medium`, `high` for thinking models

00 SCHEMA — STRUCTURED OUTPUT

Parse DSL strings into Zod schemas, validate and coerce model output.

- ✓ **DSL-to-Zod parser** — `parseDSLSchema()` parses string DSL into Zod schemas
- ✓ **Zod schema loader** — `loadZodSchema()`, `convertZodToSchema()`
- ✓ **JSON Schema conversion** — `toJSONSchema()`
- ✓ **Schema validator** — `validateSchema()`, `createValidator()`, `coerceData()`
- ✓ **Schema manager** — `SchemaManager` singleton for managing schema lifecycle

PART II — CORE ABSTRACTIONS

00 STIMULUS — PROMPT CONFIGURATION

The Stimulus defines *what* to say: persona, instructions, tools, model parameters. Pure config—doesn't run anything.

- ✓ **Stimulus class** — role, objective, instructions, reasoning, output format (`new Stimulus({ role: '...' })`)
- ✓ **Model parameter config** — temperature, maxTokens, topP, penalties; set on Stimulus, applied at runtime
- ✓ **Tool management** — `addTool`, `getTools`; tools are part of the prompt config
- ✓ **Tool instructions** — natural language instructions for tool usage
- ✓ **Skills registry** — `SkillsRegistry`, `loadSkillsFromDirectory`, `loadSkillsFromGit`
- ✓ **Skill tool** — LLM selects + loads a skill at runtime via `createSkillTool()`
- ✓ **Runner type selection** — `base` or `memory`; memory enables auto fact extraction
- ✓ **System prompt generation** — assembles system message from role + objective + instructions + context
- ✓ **Pre-built stimuli** — analysis (PDF, image, transcription), creative (cat poems, Frankenstein), coding (TypeScript, Python)

- × **Stimulus export/import** — no serialization format; can't share stimulus configs between habitats
- × **Stimulus templates/inheritance** — can't extend a base stimulus ("like that agent but with extra tools")

00 INTERACTION — CONVERSATIONS

Holds messages, model, stimulus, and runner. This is where the actual conversation lives.

- ✓ **Interaction class** — messages + model + stimulus → `chat()`, `generateText()`, `streamText()`, `generateObject()`
- ✓ **Multi-step tool calling** — `maxSteps` for iterative tool use
- ✓ **Context compaction** — `compactContext()` with pluggable strategies
- ✓ **Attachment handling** — images, files as multimodal messages
- ✓ **Normalized session format** — `NormalizedSession`, `NormalizedMessage` for cross-source compatibility
- ✓ **Session adapters** — `ClaudeCodeAdapter`, `CursorAdapter` to read external IDE sessions
- ✓ **Transcript persistence** — `onTranscriptUpdate` callback for JSONL writing
- ✓ **Checkpoint/resume** — `checkpointMessageIndex` for compaction boundaries

00 BUILT-IN TOOLS & TOOL LOADING

Tools that agents can invoke during conversations, plus a convention-based loading system.

- ✓ **URL tools** — `wget`, `markify`, `parse_feed`; fetch URLs, HTML→markdown, RSS/Atom parsing
- ✓ **PDF tools** — PDF content extraction
- ✓ **Image tools** — image analysis via LLM
- ✓ **Audio tools** — audio transcription
- ✓ **TOOL.md + handler loader** — each subdir with TOOL.md becomes a Tool; supports handler.ts, handler.js, or script execution
- ✓ **Factory-pattern handlers** — handler can export a factory function receiving context
- ✓ **Script tools** — `type: script` in TOOL.md runs external script with args
- ✓ **Math example tools** — reference implementation

00 EVALUATION

Systematic model assessment, comparison, and reporting.

✓ **EvalSuite** — declarative eval runner; VerifyTask (deterministic) + JudgeTask (LLM judge with Zod schema)

✓ **PairwiseRanker** — Elo via LLM judge; Swiss tournament + round-robin, position bias mitigation, comparison caching

✓ **Suite combine** — multi-dimension leaderboard; aggregate independent evals into unified report

✓ **Lower-level strategies** — SimpleEvaluation, MatrixEvaluation, BatchEvaluation

✓ **Result caching (two layers)** — response cache (raw LLM output) + result cache (scored output)

✓ **Multiple report formats** — console, markdown, JSON, narrative (full prose)

✓ **Dagger-based code execution** — TypeScript, Python, Ruby, Rust, Go in isolated containers

✓ **Codebase evaluation** — ContextProvider, ChangeExtractor, ChangeApplicator

✓ **CLI commands** — eval run, eval report, eval list, eval combine

✓ **Concurrency control** — `--concurrent` flag, configurable concurrency

× **Tool/skill contract tests** — no first-class framework for testing tool implementations, MCP endpoints, or OAuth flows

PART III — INTELLIGENCE

00 MEMORY — FACT EXTRACTION

Extract and store facts from conversations automatically.

✓ **Fact extraction** — LLM-powered, categorized (preference, plan, professional)

✓ **Memory operations** — ADD, UPDATE, DELETE

✓ **MemoryRunner** — auto-extract during chat (`stimulus.runnerType = 'memory'`)

✓ **Per-user fact storage** — MemoryStore interface exists

✓ **Learnings store** — `FileLearningsStore` for session-level learnings

× **Persistent memory backend** — only `InMemoryMemoryStore`; facts lost when process restarts

× **Memory retrieval/injection** — facts extracted but not automatically retrieved for future conversations

× **Cross-session memory** — each Interaction creates a fresh store; no sharing between sessions

× **Memory search/relevance** — no semantic search over stored facts

00 SESSION UNDERSTANDING & KNOWLEDGE EXTRACTION

One of the most developed areas—a full pipeline for understanding conversations and extracting reusable knowledge.

Session Persistence & Structure

✓ **JSONL transcript persistence** — append-only, per-session

✓ **Session metadata** — provider, model, type, agent

✓ **Conversation beats** — `messagesToBeats()` groups messages into logical turns

✓ **External session reading** — ClaudeCodeAdapter, CursorAdapter

✓ **Transcript resume on restart**

✓ **Segmented transcripts** — compaction creates frozen segments + live segment

Session Analysis (LLM-powered)

✓ **Session analysis via LLM** — extracts topics, tags, learnings, summary, solution type, languages, tools, success indicators

✓ **Analysis index** — `SessionAnalysisIndex` with cached results, incremental re-indexing

✓ **Session search** — keyword + field filters (tags, topic, tool, solutionType, branch, success)

✓ **Cross-project search** — discover + search across all Claude Code/Cursor project directories

Session Digest (Deep Understanding)

✓ **Session digester** — full pipeline: compaction → analysis → fact extraction → beat analysis → phase detection

✓ **Beat-by-beat breakdown** — per-beat: userRequest, toolsUsed, outcome, narrative, keyFacts

✓ **Phase detection** — groups beats into conversation phases with themes

✓ **Digest segments** — through-line + key facts per segment

✓ **Overall summary generation** — merged summary across all segments

✓ **Structured fact extraction** — typed facts from digest pipeline

✓ **Session metrics** — message count, segment count, tool calls, estimated cost, duration

✓ **Digest index** — `DigestIndex` with lightweight entries for browsing

Learnings System (Persistent Knowledge)

✓ **FileLearningsStore** — append-only JSONL per kind, per session

✓ **Learning kinds** — facts, playbooks, preferences, open_loops, mistakes

✓ **Learning provenance tracking** — links back to session, channel, compaction run

✓ **Context merge** — inject learnings into new sessions via `buildHabitatIntrospectionContextMessages()`

✓ **Compaction events** — `CompactionEventV1` marker in transcript

✓ **Session-level learnings CLI** — `sessions digest` subcommands

Session Understanding Gaps

⚠ **Cross-session learning injection** — can inject from same session's store, but no automatic pull from all past sessions

✕ **Semantic search over learnings** — stored but can only be loaded in bulk, not searched by relevance

✕ **Cross-platform session continuity** — can't continue a Discord conversation on Telegram or web

✕ **Multi-instance session safety** — no distributed locks; multiple workers would corrupt

✕ **Automatic digest on session end** — digest is manual only

✕ **Learning deduplication/consolidation** — learnings accumulate forever, no merging

00 CONTEXT MANAGEMENT

Track context size and compact long conversations.

✓ **Context compaction** — pluggable `CompactionStrategy` interface

✓ **Token estimation** — `estimateContextSize()`

✓ **Compaction segment detection** — `getCompactionSegment()`

✓ **Strategy registry** — `registerCompactionStrategy()` / `getCompactionStrategy()`

PART IV — THE AGENT PLATFORM

00 HABITAT — AGENT CONTAINER

The top-level system. Manages work directory, config, sessions, tools, agents, and secrets.

- ✓ **Work directory with config.json**
- ✓ **Agent secrets** — secrets.json, 0600 perms (plain JSON, no encryption at rest)
- ✓ **Tool registration via ToolSet interface** — 10 standard tool sets registered by default
- ✓ **Work-dir tools** — TOOL.md + handler files
- ✓ **Skills from directory or Git repos**
- ✓ **Self-modify tools** — create/remove/reload tools at runtime
- ✓ **Stimulus from work dir files** — STIMULUS.md, README, CLAUDE.md
- ✓ **Memory files loading** — `memoryFiles` config with journal support
- ✓ **Sub-agents** — managed projects with agent CRUD
- ✓ **Agent discovery** — MCP health monitoring
- ✓ **Onboarding wizard**
- ✓ **Gaia HTTP server** — JSON API + chat (sessions, beats, commands, chat endpoints)
- ✓ **Session management** — create, list, resume, metadata; per-type (CLI, Discord, Telegram, web, API)
- ✓ **Interaction factory** — `createInteraction()` with auto-session and transcript persistence
- ✓ **Ephemeral agent interactions** — `createAgentInteraction()` for diagnosis/monitor agents
- ✓ **Work dir file management** — `readWorkDirFile()`, `writeWorkDirFile()`, state files

Habitat Tool Sets

✓ **File operations** — read_file, write_file, list_directory, ripgrep (sandboxed to allowed roots)

✓ **Time** — current_time

✓ **URL operations** — wget, markify, parse_feed

✓ **Agent management** — list/add/update/remove agents

✓ **Session management** — list/show/messages/stats/inspect/read_file, learnings, transcript compact

✓ **External interactions** — read Claude Code/Cursor history

✓ **Agent runner** — agent_clone, agent_logs, agent_status, agent_ask, bridge_diagnose, bridge_monitor

✓ **Secrets** — set/remove/list secrets

✓ **Search** — web search via Tavily (needs TAVILY_API_KEY)

✓ **Self-modify** — create/remove/reload tools and skills at runtime

✓ **Discord routing** — channel→agent binding tools

Habitat Bridge (Containerized Sub-Agents)

✓ **BridgeAgent** — containerized execution via Dagger

✓ **BridgeSupervisor** — lifecycle management

✓ **MCP SDK server/client** — StreamableHTTP transport

✓ **Go MCP server implementation** — alternative server for bridges

✓ **LLM-driven diagnosis agent** — read-only project inspection and provisioning detection

✓ **LLM-driven monitor agent** — container health monitoring

✓ **Persistent logging**

✓ **Secret injection into containers**

✓ **Port pool management** — auto-assigned MCP ports

✓ **Bridge state persistence** — `BridgeState` saved/loaded per agent

Habitat Gaps

× **Per-user credentials on Habitat** — mcp-serve has per-user upstream tokens, but core Habitat only has agent-level secrets

× **Stimulus sharing between habitats** — no way to export/import a stimulus config

× **Secrets encryption at rest** — secrets.json is plaintext

× **Secret scoping** — all secrets are habitat-scoped; no user-level or global-level

× **Config validation / habitat doctor** — no schema validation, env var checks, or diagnostic command

00 MCP — MODEL CONTEXT PROTOCOL

Full MCP implementation on both client and server sides, plus a library for building hosted multi-user MCP servers.

✓ **Remote MCP client with OAuth PKCE** — `RemoteMcpClient` with file-backed token storage at `~/.umwelten/mcp-auth/`

✓ **MCP client (custom protocol)** — low-level `MCPClient` for stdio/HTTP transports

✓ **MCP server** — expose umwelten as tool provider

✓ **MCP→Stimulus bridge** — load MCP tools into a Stimulus, convert to Vercel AI SDK tools

✓ **mcp chat CLI** — connect + REPL or one-shot, with `--logout` for credential reset

✓ **mcp connect and mcp test-tool CLI**

✓ **mcp-serve library** — generic "upstream OAuth → hosted MCP server" toolkit

✓ **NeonStore** — Postgres persistence for mcp-serve

✓ **UpstreamOAuthProvider interface** — implement for any OAuth service

✓ **McpToolRegistrar** — per-user tool registration

✓ **Oura Ring MCP example** — rebuilt on mcp-serve library

✓ **Twitter MCP example** — 12 tools, per-user Twitter OAuth

× **Per-tool authorization/scopes** — MCP auth is all-or-nothing; no per-tool read/write permissions

× **Audit logs** — no record of who called what tool, when, with whose credentials

× **Admin surface for hosted MCPs** — no way to list users, revoke auth, inspect usage without DB access

× **Per-user quotas/rate limits/billing** — cost tracking exists per model call, but no per-user metering on MCP server

× **Token revocation endpoint** — MCP tokens can't be explicitly revoked by the user

× **Skill→hosted MCP packaging flow** — missing skill manifest format, deploy workflow

00 MULTI-PLATFORM INTERFACES

Agents accessible from multiple surfaces through a unified ChannelBridge routing layer.

✓ **CLI REPL**
✓ **Discord adapter** — ambient mode, threads, slash commands, attachment handling
✓ **Telegram adapter** — media storage, message logging
✓ **Web interface (Gaia server)** — JSON API + chat endpoint
✓ **TUI (React Ink)**
✓ **ChannelBridge** — unified adapter layer; all platforms share one bridge
✓ **Unified routing** — routing.json → agent binding with runtime mode selection
✓ **Unified slash commands** — /switch, /status, /reload-routing, etc.
✓ **Claude SDK pass-through** — claude-sdk runtime
✓ **Transcript resume on restart**

× **Gaia web server authentication** — no auth, no sessions, no user identity
⚠ **Consistent multimodal output** — input attachments work; output is text-only across all adapters
× **Platform capability negotiation** — bridge doesn't know what each platform supports

00 IDENTITY & ACCESS CONTROL

Current identity model is functional but minimal.

✓ **userId field on Interaction** — forwarded to OpenRouter/Anthropic for analytics

✓ **Per-user memory** — MemoryStore keyed by userId (but in-memory only)

✓ **MCP OAuth for user identity** — in mcp-serve layer

× **Unified user identity across channels** — Discord user ≠ Telegram user ≠ MCP user; no way to link them

× **User account management** — no signup, login, profile, linked accounts

× **Per-user credential store on Habitat** — no `getUserSecret(userId, service)`

× **Role-based access control** — no admin vs user vs readonly roles

× **Data lifecycle/privacy controls** — no delete-my-data, retention rules, export, redaction

00 BACKGROUND & AUTOMATION

The least developed area. Habitats can only react to incoming chat messages.

× **Webhook/event ingestion** — no HTTP webhook handler, no event bus, no “when X happens, do Y”

× **Scheduled tasks/cron** — no way to say “every morning, summarize my Twitter feed”

× **Background job runtime** — no job queue, retries, idempotency, or failure handling

⚠ **Inter-habitat communication** — sub-agents exist, bridge delegates via MCP, but no direct tool-to-tool calls or pub/sub

PART V — DEVELOPER EXPERIENCE

00 URL & HTML UTILITIES

Low-level utilities consumed by the tool layer.

- ✓ **URL fetching with limits** — `fetchUrl()` with timeout and size limits
- ✓ **HTML→Markdown (Turndown)** — `fromHtmlBuiltIn()`
- ✓ **HTML→Markdown (Markify service)** — `fromHtmlViaMarkify()`, optional external service
- ✓ **URL→Markdown pipeline** — `urlToMarkdown()` fetch + convert
- ✓ **RSS/Atom feed parsing** — `parseFeed()`

00 REPORTING

Unified report rendering used by tool tests and evaluation suite reports.

- ✓ **Unified Reporter class** — `fromToolTest()`, `toConsole()`, `toMarkdown()`, `toHtml()`, `toJson()`
- ✓ **Console renderer** — formatted terminal output
- ✓ **Markdown renderer** — structured markdown reports
- ✓ **HTML renderer**
- ✓ **JSON renderer**
- ✓ **Report types** — tool-test, code-generation, evaluation, batch, suite
- ✓ **Narrative reports** — full methodology + analysis + cost/speed breakdown
- ✓ **Report adapters** — adapt different result types to unified Report format

00 CLI

Commander-based CLI providing the primary developer interface.

✓ **models** — list/search across providers (`--search`, `--provider`, `--json`)

✓ **run** — one-shot prompt

✓ **chat** — interactive REPL

✓ **eval** — run, report, list, combine subcommands

✓ **sessions** — list, index, search, browse, digest

✓ **habitat** — agent REPL + interfaces (telegram, discord, web)

✓ **mcp** — chat, connect, test-tool

✓ **telegram** — standalone Telegram bot

✓ **tools** — list available tools

✓ **Common options** — `--provider`, `--model`, `--temperature`, etc. shared across commands

00 DEVELOPER EXPERIENCE

Documentation, tooling, and developer workflow support.

✓ **CLAUDE.md** — maintainer module map

✓ **LLM.txt** — agent-friendly summary

✓ **VitePress documentation site**

✓ **Examples** — evals, oura-mcp, twitter-mcp, habitat-minimal, and more

✓ **mise.toml tasks**

⚠ **Hot-reload of tools/skills** — self-modify tools exist but no automatic watch-mode

× **Skill versioning/compatibility** — no version constraints, no migration, no “requires umwelten ≥0.5”

× **Skill manifest format** — SKILL.md is human-readable but not machine-parseable

× **MCP server smoke/integration tests** — no test harness for end-to-end verification

× **Safety/approval model for dangerous tools** — no dangerous-tool classification or approval hooks

× **Observability dashboard** — cost data exists in pieces but no unified view or alerts

00 EXAMPLES

Working reference implementations in the `examples/` directory.

- ✓ **examples/evals/** — EvalSuite examples: reasoning (LLM judge), instruction (deterministic), car-wash
- ✓ **examples/model-showdown/** — 5-dimension eval suite + combine + reports
- ✓ **examples/oura-mcp/** — multi-user Oura Ring MCP server (mcp-serve + Neon + fly.io)
- ✓ **examples/twitter-mcp/** — multi-user Twitter MCP server (mcp-serve + Neon + fly.io)
- ✓ **examples/habitat-minimal/** — smallest Habitat work-dir layout
- ✓ **examples/jeeves-bot/** — Discord/Telegram bot example
- ✓ **examples/mcp-chat/** — PairwiseRanker + MCP tool-use responses
- ✓ **examples/memorization/** — memory extraction demo
- ✓ **examples/provider-comparison/** — multi-provider comparison
- ✓ **examples/schemas/** — structured output examples
- ✓ **examples/gaia-ui/** — Gaia web interface

00 PRIORITY GAPS FOR THE VISION

> The "turn any skill into a hosted MCP with per-user access control" vision is partially realized with `mcp-serve`. The critical remaining gaps:

Tier 1 — Enables the Multi-User Agent Platform

- **Unified user identity** — same person across Discord, Telegram, MCP, web
- **Per-user credential store on Habitat** — `habitat.getUserSecret(userId, 'twitter')`
- **Persistent memory backend** — facts survive restarts, shared across sessions
- **Memory retrieval loop** — extract → store → retrieve → inject as context

Tier 2 — Enables Production Hosting

- **Gaia web auth** — at minimum, API key or session-based auth
- **Audit logs** — who called what, when, with whose credentials
- **Per-user quotas/metering** — track and cap usage per user
- **Secrets encryption** — at rest, with rotation support

Tier 3 — Enables the Skill Marketplace

- **Skill manifest format** — machine-readable declaration of auth, scopes, env vars, tools
- **Stimulus portability** — export/import agent personality and tool config
- **Skill versioning** — semver, compatibility constraints, migration
- **Packaging + deploy workflow** — `umwelten publish` → hosted MCP with OAuth

Tier 4 — Enables Autonomous Agents

- **Scheduled tasks/cron** — agents that act on their own schedule
- **Webhook ingestion** — agents that react to external events
- **Background job runtime** — queues, retries, failure handling